



University of
Nottingham

UK | CHINA | MALAYSIA

COMP4139

Machine Learning (MLE)

Generative Models

Dr Shreyank N Gowda
Dr Direnc Pekaslan

School of Computer Science
University of Nottingham



- Generative and Discriminative Models
- Autoencoders
- Variational Autoencoders
- Generative Adversarial Networks
- Applications of GANs
- Diffusion Models



Generative Models vs Discriminative Models

Discriminative Models



Features Class

$X \rightarrow Y$

Generative Models



Noise Class Features

$\epsilon, Y \rightarrow X$

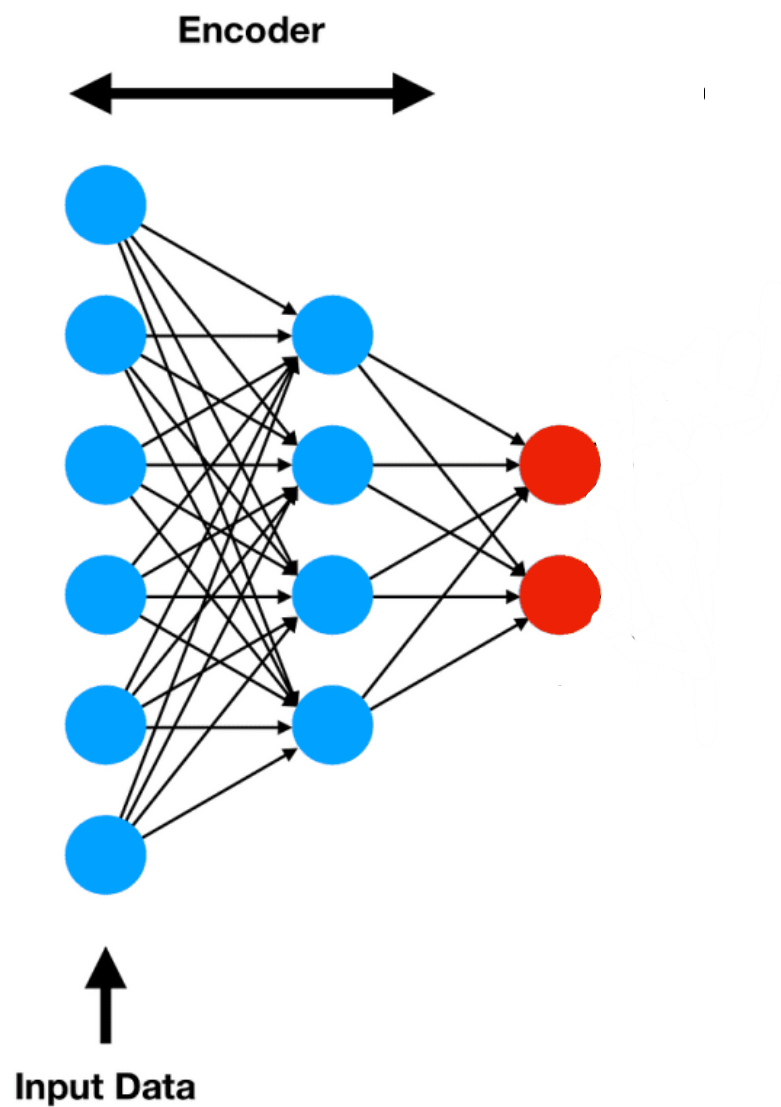
Generative Models vs Discriminative Models

- Discriminative models distinguish different data instances.
 - Learns to capture the conditional probability $p(Y|X)$. X: input, Y: label
- Generative models can generate new data instances.
 - Learns to capture the probability distribution $p(X)$ of the data itself. X: input
- Generative models tackle a much harder task than discriminative models.
 - Distinguishing things is easier than **generating things** Key aim of generative models

- Three methods stand out:
 - Variational Autoencoders, Generative Adversarial Networks and Diffusion Models.

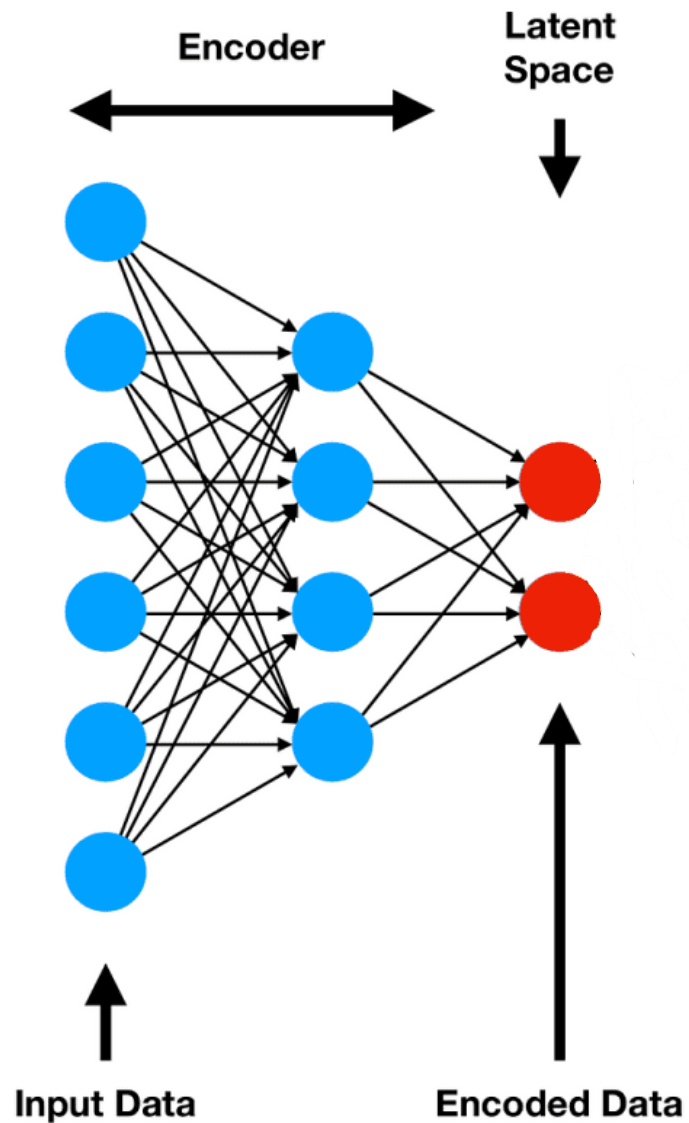


Autoencoders



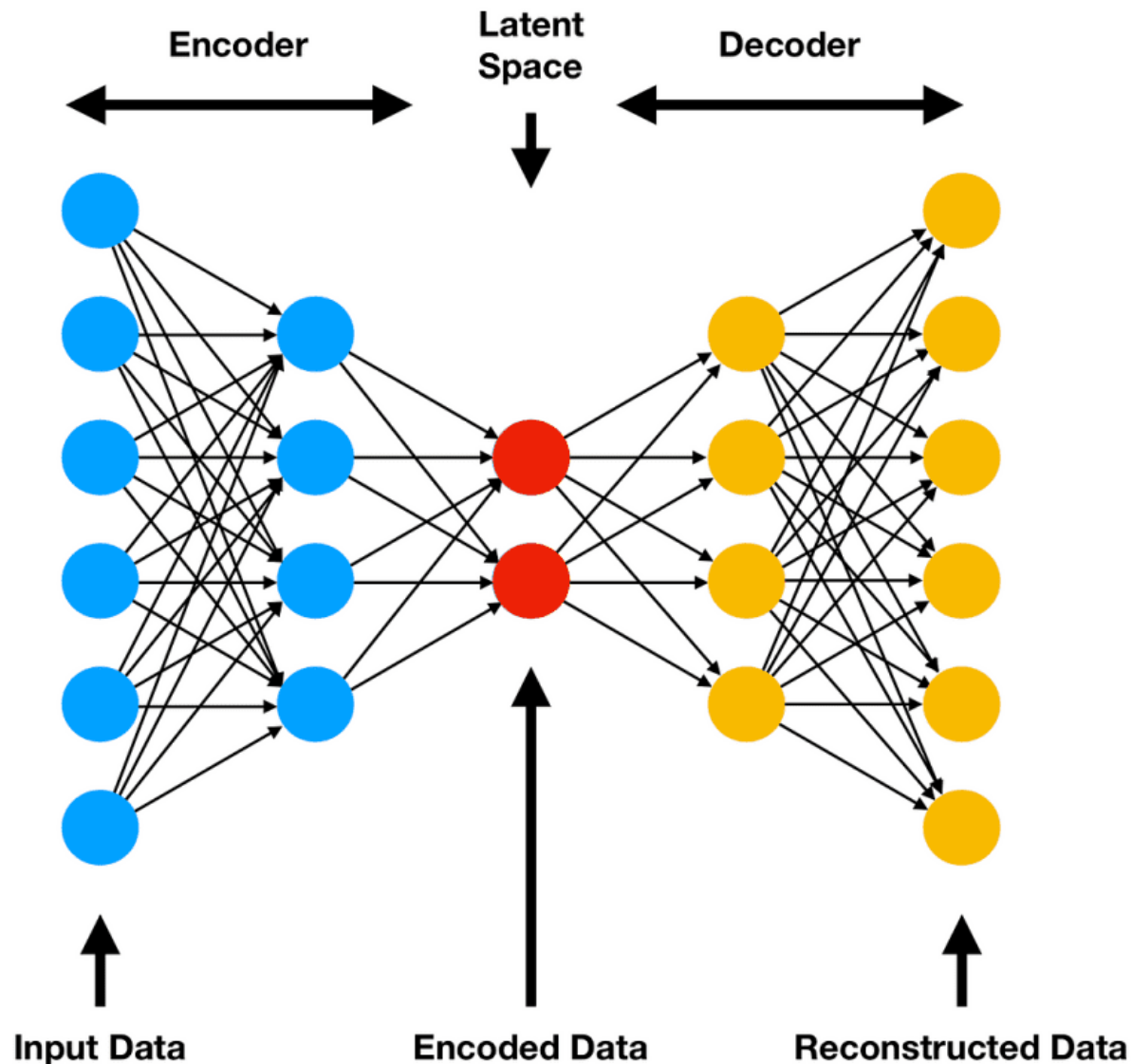


Autoencoders



- Unsupervised
- Learn lower dimensional features
- Dimensionality Reduction!

Autoencoders

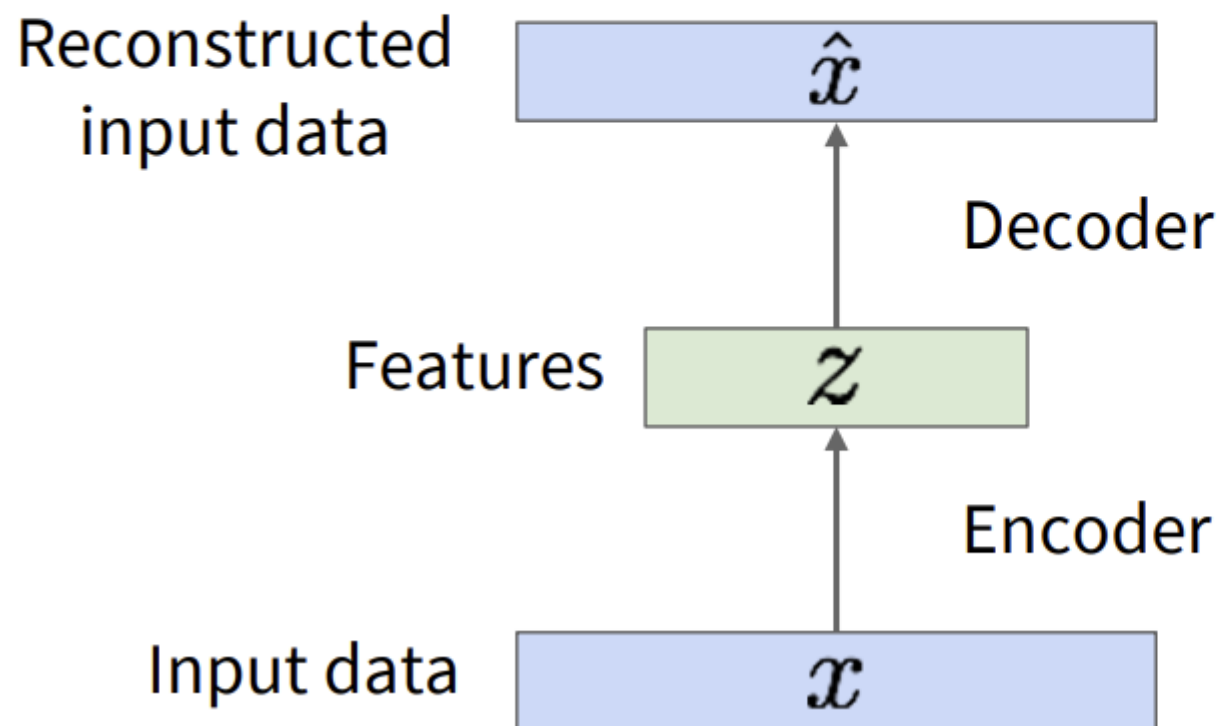


- Unsupervised
- Learn lower dimensional features
- Dimensionality Reduction!

Train such that features can be used to reconstruct original data
“Autoencoding” - encoding input itself



Autoencoders



Autoencoders can reconstruct data, and can learn features to initialize a supervised model

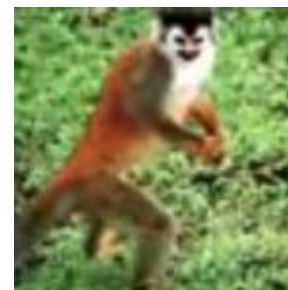
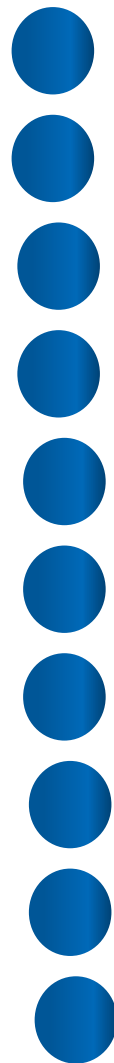
Features capture factors of variation in training data.

But we can't generate new images from an autoencoder because we don't know the space of z .

How do we make autoencoder a generative model?



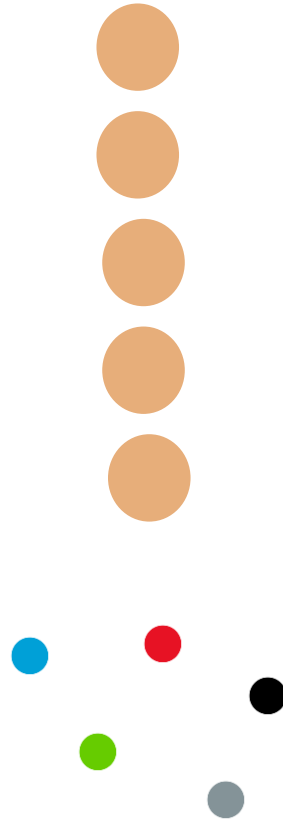
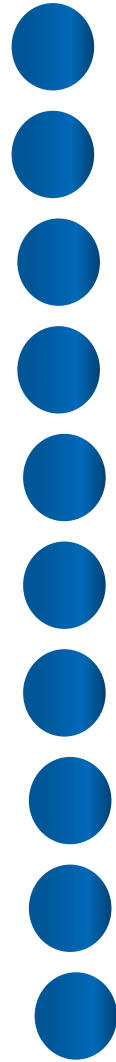
Latent Space of Autoencoders





Latent Space of Autoencoders

A
B
C
D
E

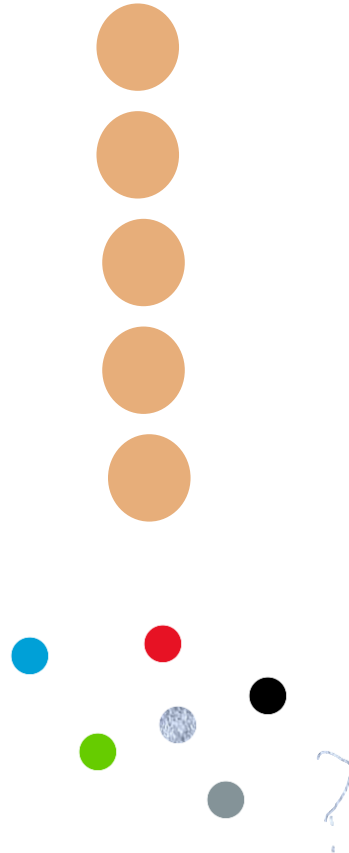
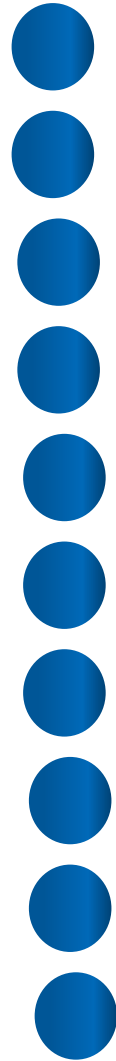


A
B
C
D
E



Latent Space of Autoencoders

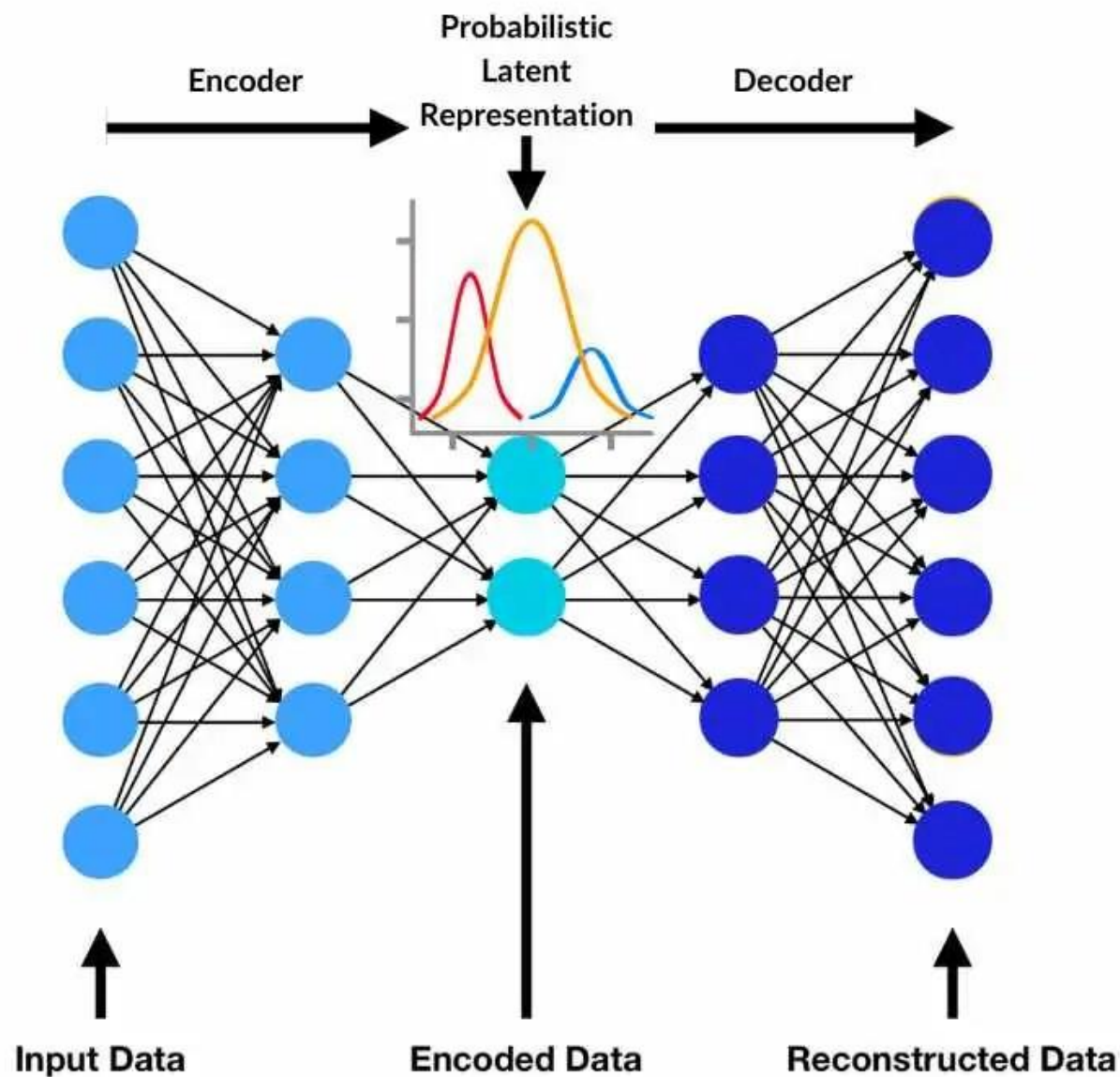
A
B
C
D
E



A
B
C
D
E
~~Handwritten scribbles~~

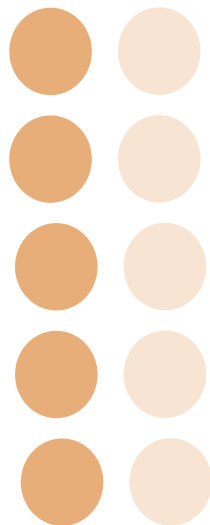
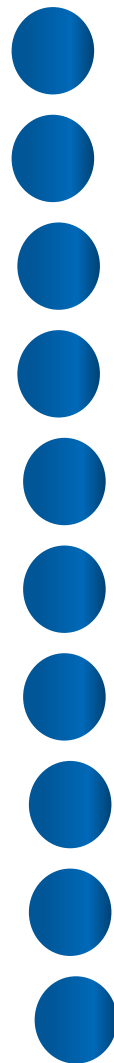


Variational Autoencoders





Latent Space of Variational Autoencoders

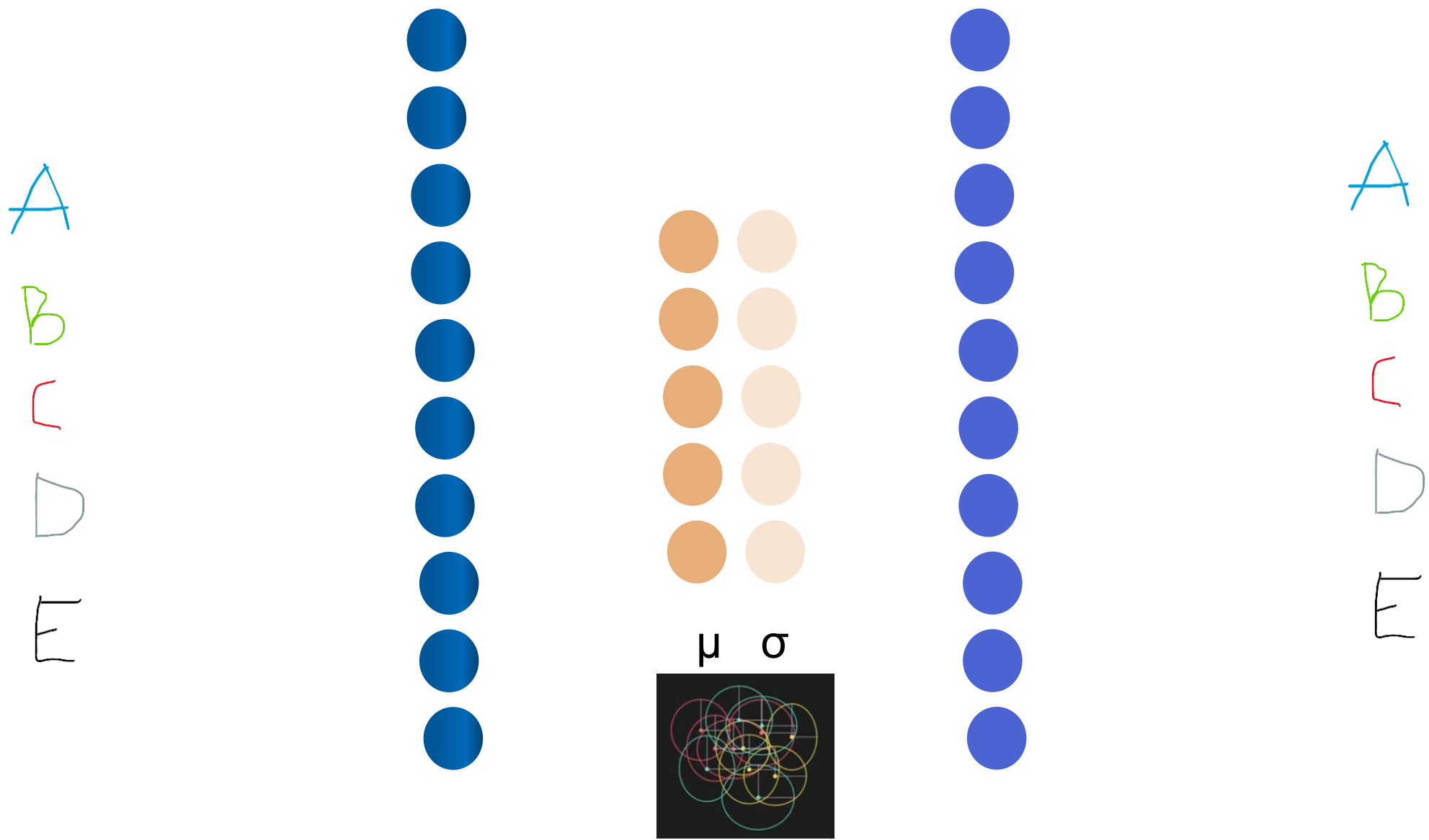


μ σ





Latent Space of Variational Autoencoders





Latent Space of Variational Autoencoders

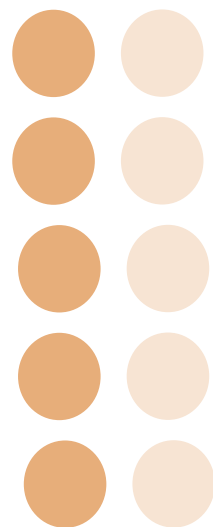
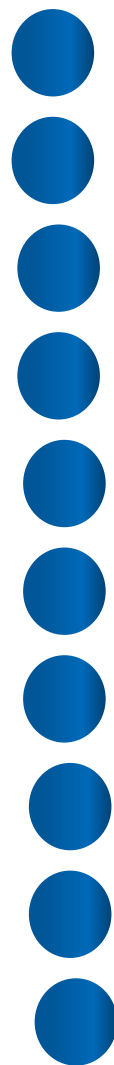
A

B

C

D

E



μ σ



A

B

C

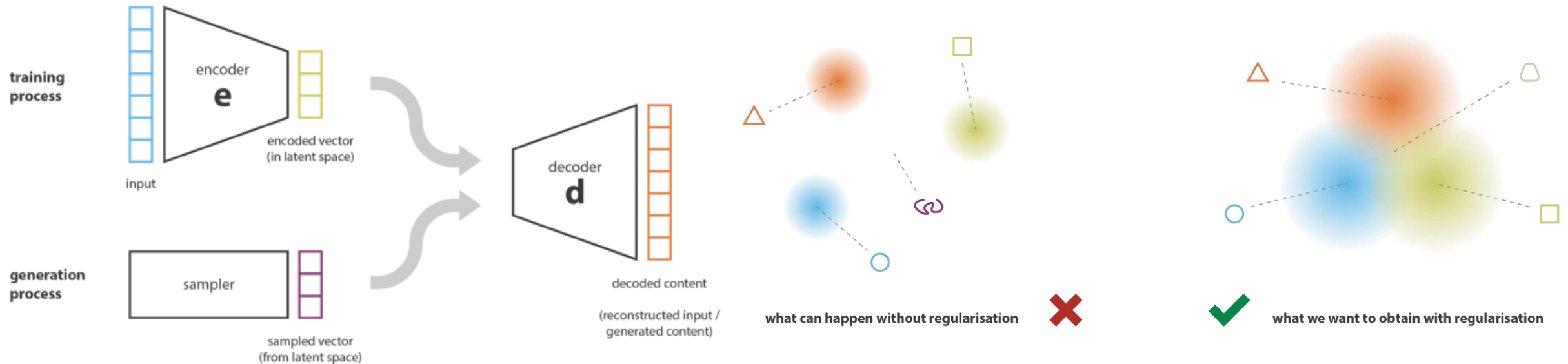
D

E

F

Generate New Examples Using the Decoder

- Once the encoder-decoder is ready trained. The decoder can be used to generate new examples by sampling the learned latent space.
- However, how to sample the latent space?

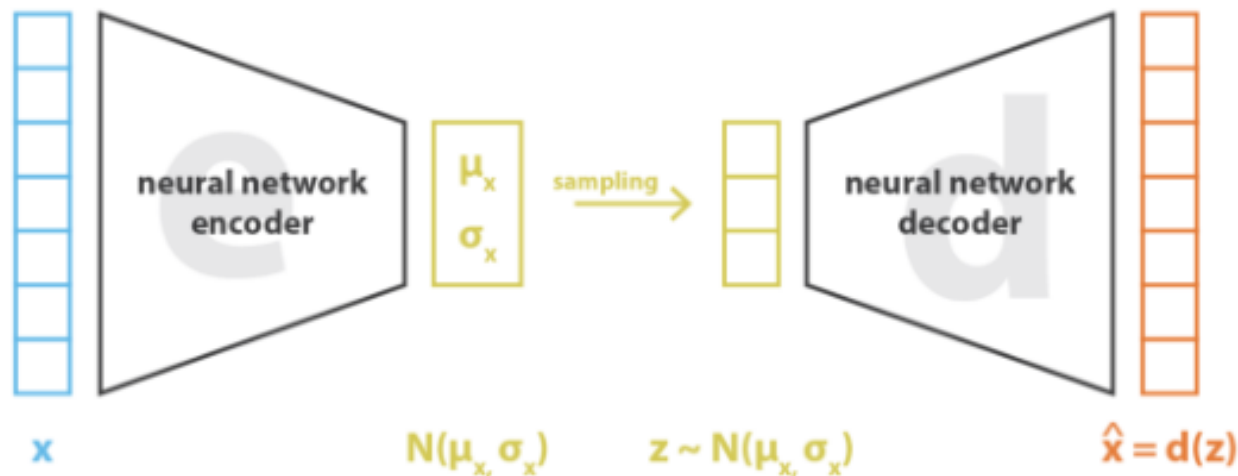


Loss Function in VAE

The loss function contains two terms:

- Reconstruction term: makes the generated output as similar as to the input.
- Regularisation term: makes the learned latent features from the encoder as close as to a Gaussian distribution using Kulback-Leibler (KL) divergence measure.

$$L = \frac{1}{n} \|x - \hat{x}\|^2 + \alpha KL[N(\mu_x, \sigma_x), N(0, I)]$$



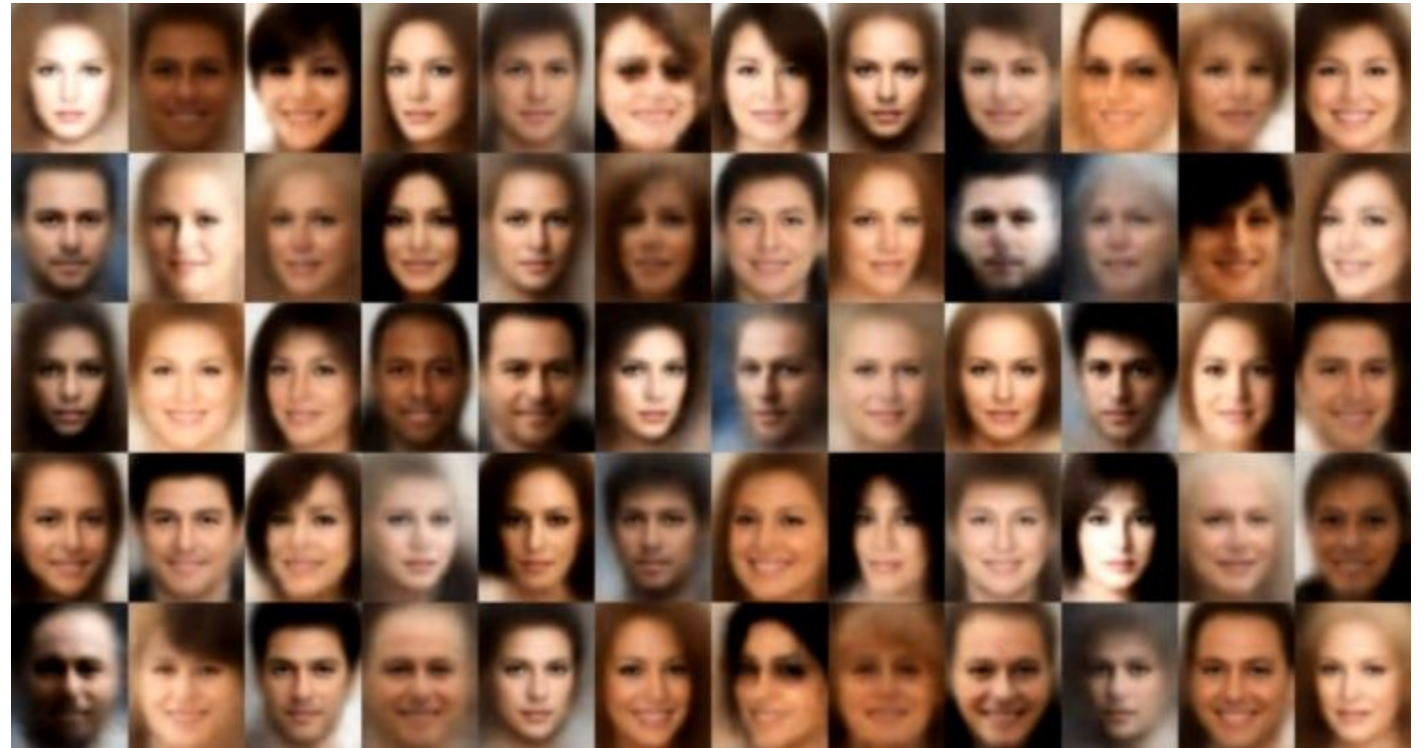


Regularisation in Variational Autoencoders (VAEs)

- Without regularisation, the encoder-decoder model can be **overfitted** to the input data and reconstruct very accurate examples of the training data but **can't generate** realistic new examples.
- In VAE, the encoder learns to map the input data into a **Gaussian (normal) distribution** (contains mean and covariance matrix).
- So that we can then sample from this normal distribution to generate new examples using the decoder.

VAE Inference

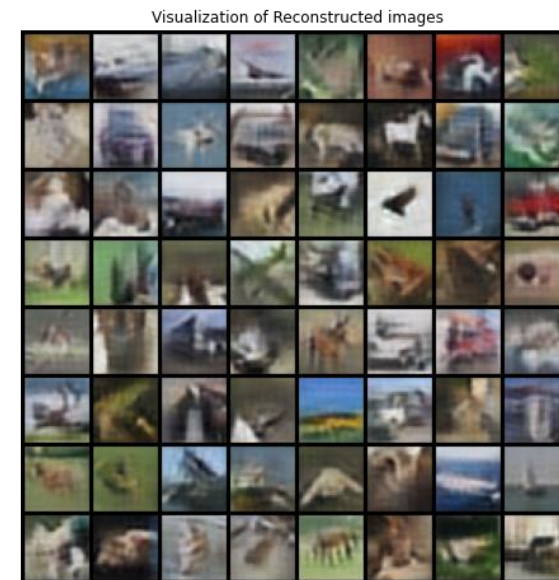
- New examples can be generated by sampling the learned mean and covariance then inputting to the decoder.
- There is a trade-off between the regularisation and reconstruction loss.
 - Larger regularisation makes the reconstruction error larger, hence reconstructed data is less realistic (e.g. image blurry).



Face images generated with a Variational Autoencoder
(source: Wojciech Mormul on Github).

Advantages and Disadvantages

- Advantages:
 - Learned latent space is well constrained and easy to sample from.
 - Easier to train compared to other generative models: smaller training set.
 - Broad application scenarios: e.g. anomaly detection, data synthesis, etc.
- Disadvantages:
 - Generated images tend to be blurry.





Introduction to Autoencoders

Generative Adversarial Network (GAN)

- “Generative Adversarial Networks is the most interesting idea in the last 10 years in Machine Learning.” — Yann LeCun, Director of AI Research at Meta
- Very broad applications: super-resolution, image in-painting, image synthesis, text-to-image, etc.



Face Synthesis



Sketch to image



Image in-painting

Problem: Want to sample from complex, high-dimensional training distribution. No direct way to do this!

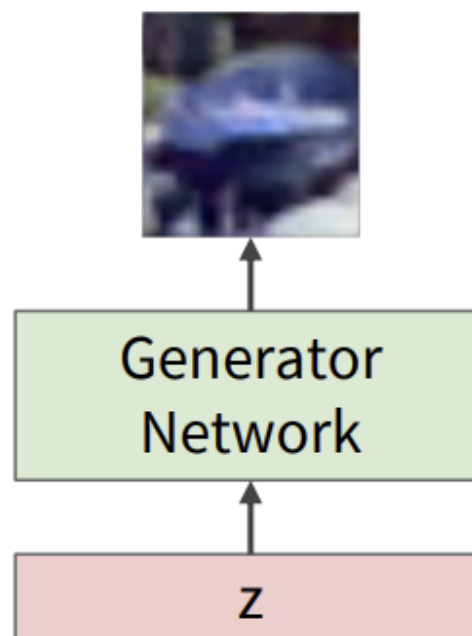
Solution: Sample from a simple distribution we can easily sample from, e.g. random noise. Learn transformation to training distribution.

But we don't know which sample z maps to which training image $\rightarrow$ can't learn by reconstructing training images

Output: Sample from training distribution

Objective: generated images should look "real"

Input: Random noise





Problem: Want to sample from complex, high-dimensional training distribution. No direct way to do this!

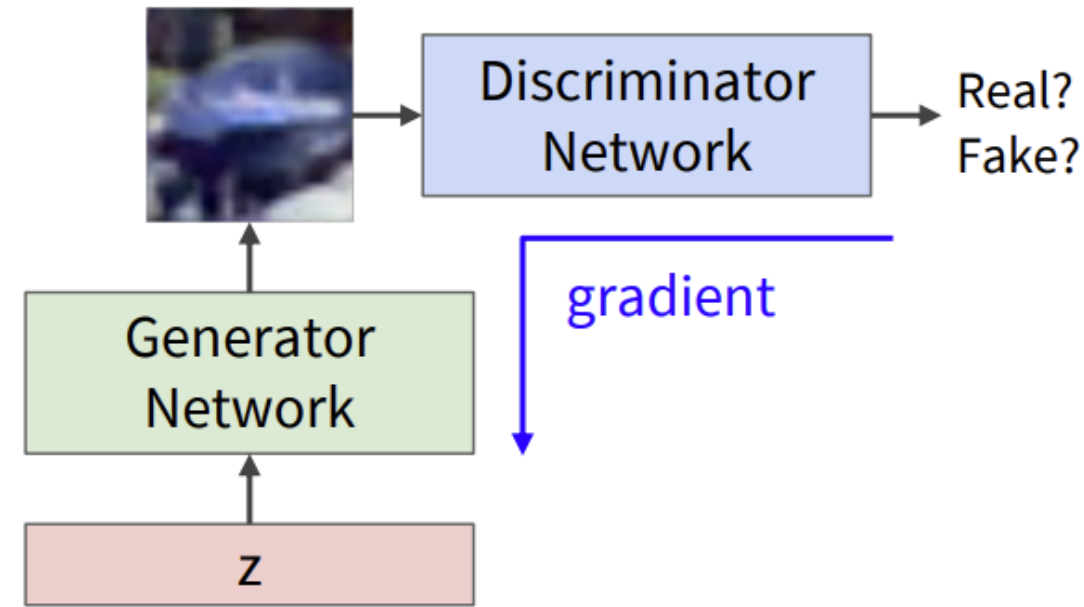
Solution: Sample from a simple distribution we can easily sample from, e.g. random noise. Learn transformation to training distribution.

But we don't know which sample z maps to which training image $\rightarrow$ can't learn by reconstructing training images

Solution: Use a discriminator network to tell whether the generated image is within data distribution ("real") or not

Output: Sample from training distribution

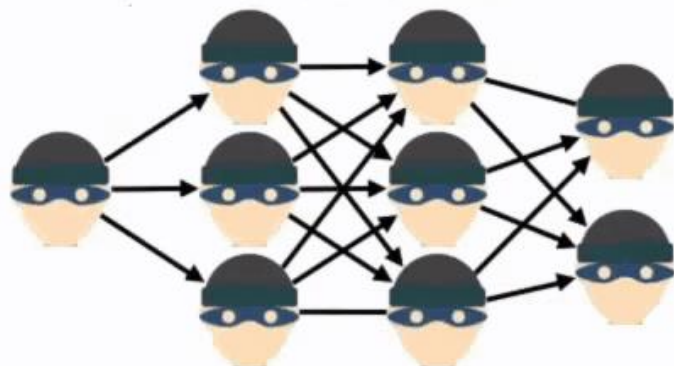
Input: Random noise



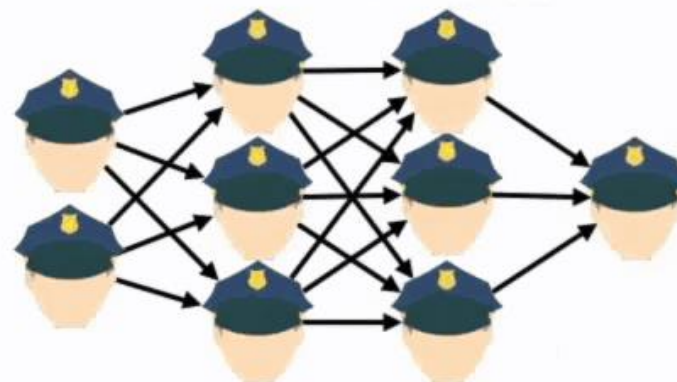


GANs

Generator



Discriminator

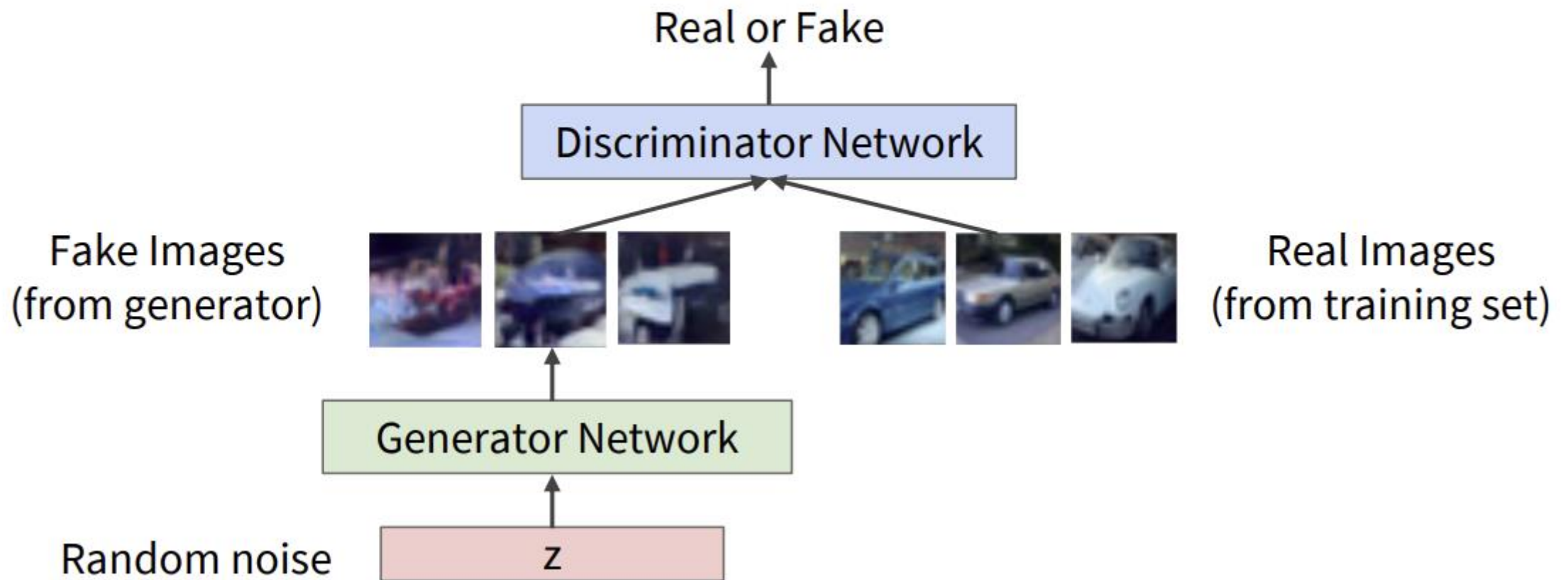


Real images



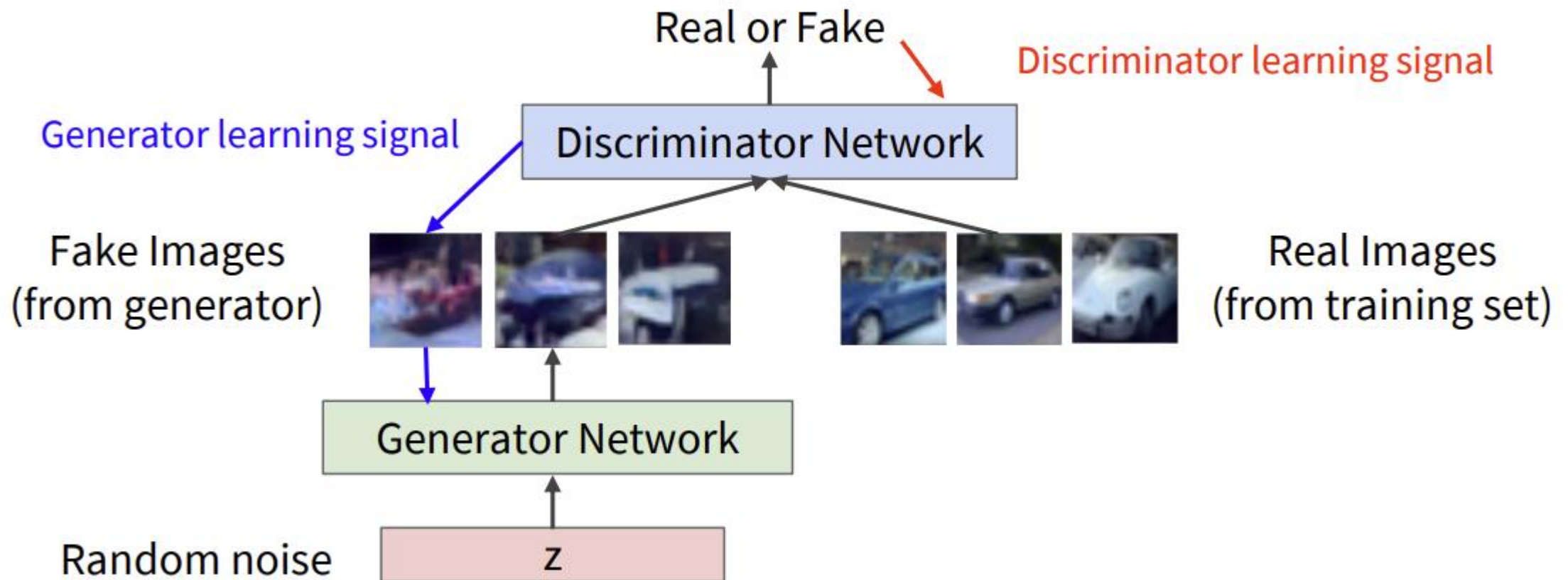
Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images



Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images



Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **minimax game**

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Generator objective

Discriminator objective

Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images

Train jointly in minimax game

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log \underbrace{D_{\theta_d}(x)}_{\substack{\text{Discriminator output} \\ \text{for real data } x}} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\substack{\text{Discriminator output for} \\ \text{generated fake data } G(z)}}) \right]$$

- Discriminator (θ_d) wants to maximize objective such that $D(x)$ is close to 1 (real) and $D(G(z))$ is close to 0 (fake)
- Generator (θ_g) wants to minimize objective such that $D(G(z))$ is close to 1 (discriminator is fooled into thinking generated $G(z)$ is real)

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. Gradient ascent on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

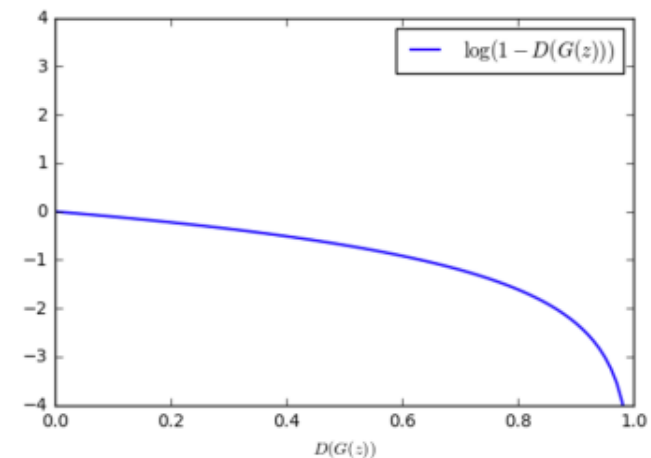
2. Gradient descent on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

In practice, optimizing this generator objective does not work well!

When sample is likely fake, want to learn from it to improve generator (move to the right on X axis).

- Generator to maximise: $\mathbb{E}_z[\log(D(G(z)))]$

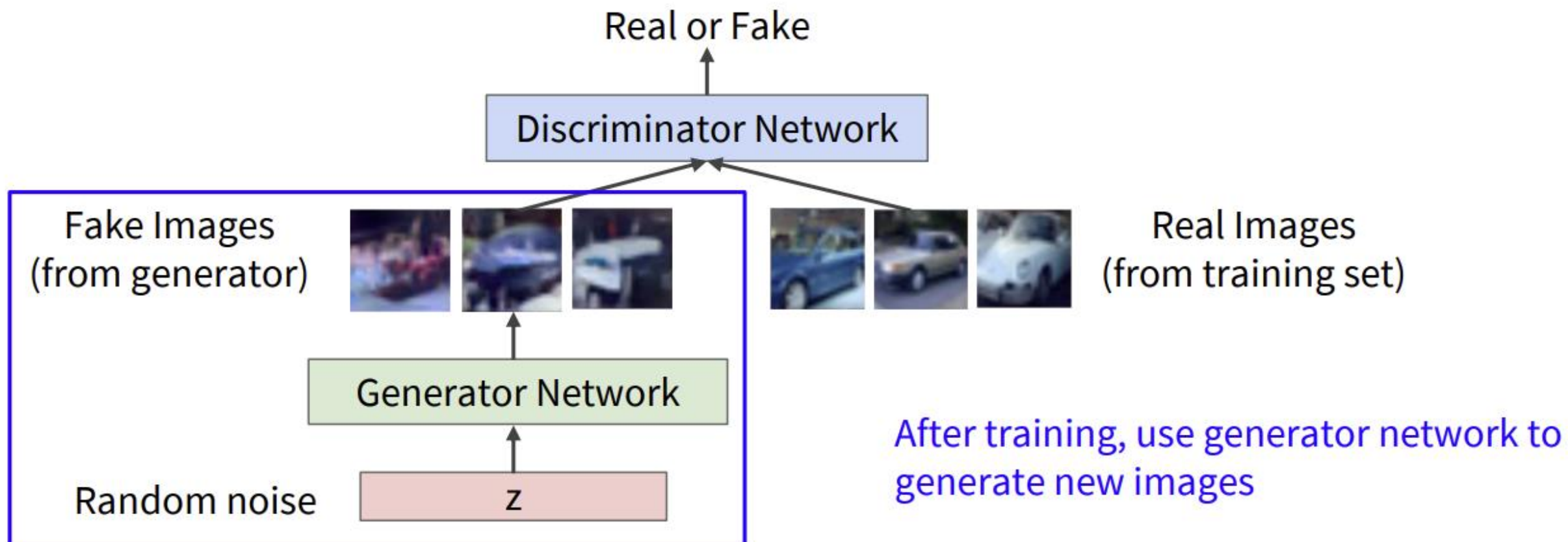




Training GANs

Generator network: try to fool the discriminator by generating real-looking images

Discriminator network: try to distinguish between real and fake images





Discriminator

- The discriminator is simply a classifier, which could use any network architecture appropriate to the type of data it's classifying (e.g. VGG)
 - Real data: from real samples in the training set
 - Fake data: synthesised data from the generator
- During discriminator training:
 - The discriminator classifies both real data and fake data.
 - The discriminator loss penalises the discriminator for **misclassifying a real and a fake** instance.
 - The discriminator updates its weights through backpropagation from the discriminator loss through the discriminator network.



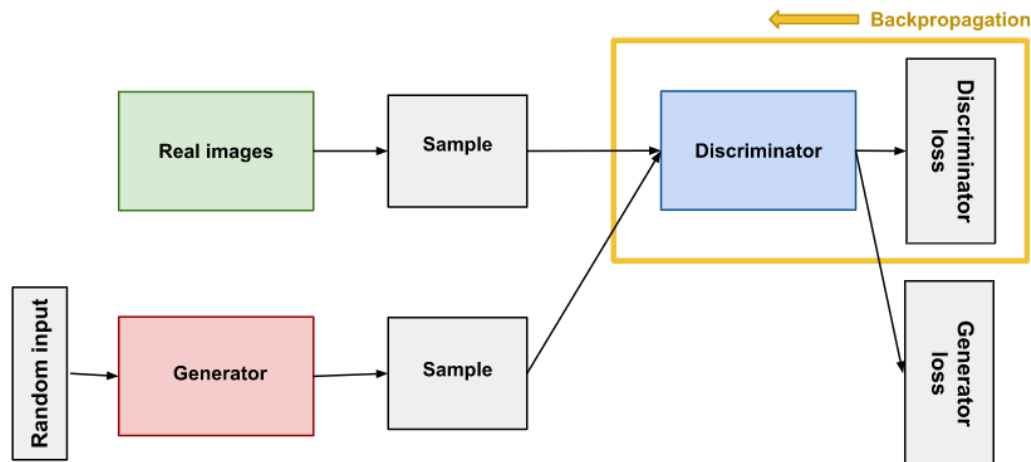
- The generator part of a GAN learns to create fake data by incorporating feedback from the discriminator. It learns to make the discriminator classify its output as real.
- During generator training:
 - Input random noise (e.g. vector in a small dimension sampled from uniform distribution)
 - The input goes through the generator network to generate a data instance
 - The discriminator classifies the generated data with an output
 - Calculate the loss from discriminator classification, which penalises the generator for **failing to fool** the discriminator based on the output
 - Backpropagate through both the discriminator and generator to obtain gradients
 - Use gradients to change only the generator weights **but don't update the discriminator**

Training Discriminator and Generator

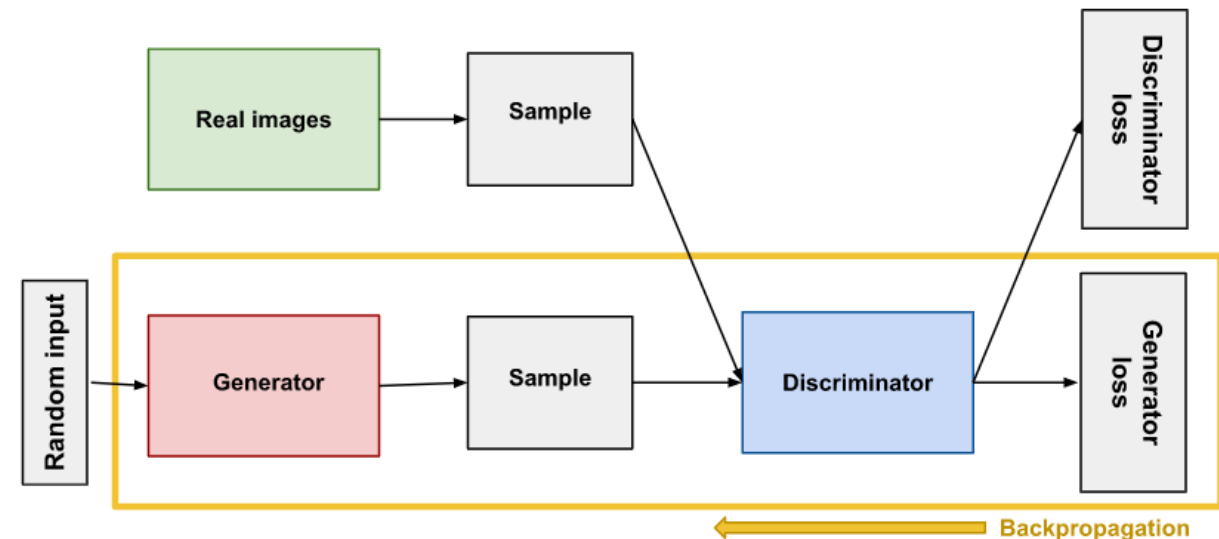
GAN's iterative training process:

- The discriminator trains for one or more epochs.
- The generator trains for one or more epochs.
- Alternate the above two steps to continue to train the generator and discriminator networks.
- The generator improves with training and the discriminator performance gets worse. If the generator succeeds perfectly, then the discriminator has a 50% accuracy.
- **The discriminator feedback gets less meaningful over time makes GAN training unstable.**

Discriminator learning



Generator learning





Loss Functions

- The loss functions that reflect the distance between the distribution of the data generated by the GAN and the distribution of the real data.
- Original GAN loss:
 - Discriminator to maximise: $E_x[\log(D(x))] + E_z[\log(1 - D(G(z)))]$
 - Generator to maximise: $E_z[\log(D(G(z)))]$
 - $D(x)$: discriminator's estimate of the probability that real data x is real
 - E_x : the expected value over all real data samples.
 - $G(z)$: the generator's output when given noise z .
 - $D(G(z))$: the discriminator's estimate of the probability of a fake example is real
 - $E(z)$: the expected value over all random inputs to the generator (all fake examples)



Advantages of GANs

- Superior output quality: GANs are known for producing sharper, more realistic, and high-fidelity outputs, especially in image generation, where VAEs can often result in blurry images.
- Unrestricted latent variable size: GANs do not impose restrictions on the size of the latent variable, which allows for more adaptability in modeling complex data distributions.

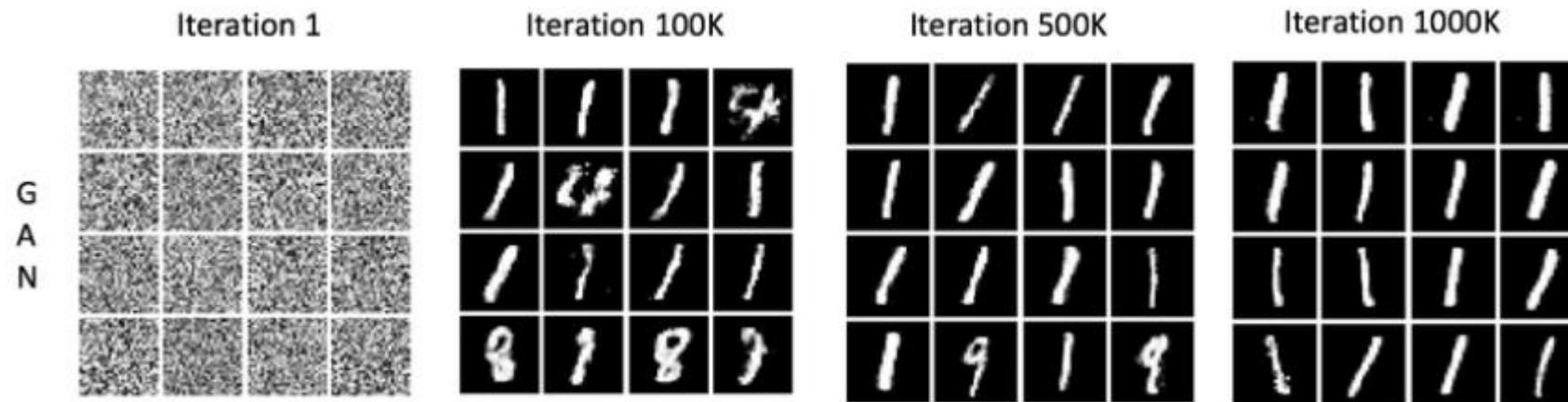


Problems of GAN

- Tough to train:
 - If the discriminator behaves badly, the generator does not have accurate feedback and the loss function cannot represent the reality.
 - If the discriminator does a great job, the gradient of the loss function drops down to close to zero and the learning becomes super slow or even jammed.

Problems of GAN

- Mode Collapse:
 - During the training, the generator may collapse to a setting where it always produces similar outputs with low variety.

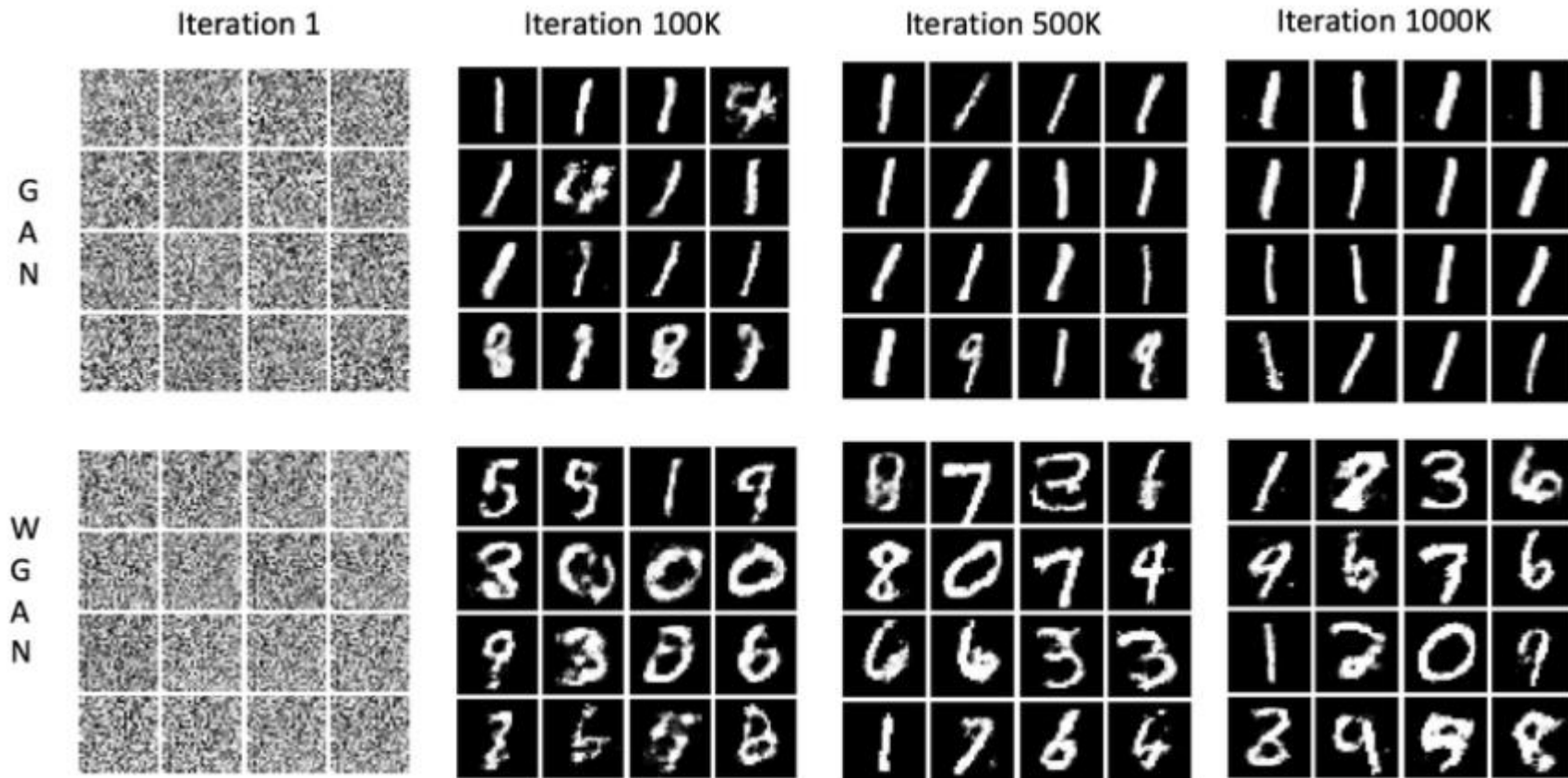


Improving GAN

- Tricks to improve GAN:

- Use Wasserstein distance to replace KL divergence: discriminator no longer discriminate between real and fake but calculate a smooth distance of their probability distribution.

<https://lilianweng.github.io/lil-log/2017/08/20/from-GAN-to-WGAN.html>



Comparing outputs

Query



Prominent attributes: White, Fully Visible Forehead, Mouth Closed, Male, Curly Hair, Eyes Open, Pale Skin, Frowning, Pointy Nose, Teeth Not Visible, No Eyewear.

VAE



GAN



VAE/GAN



Query



Prominent attributes: White, Male, Curly Hair, Frowning, Eyes Open, Pointy Nose, Flash, Posed Photo, Eyeglasses, Narrow Eyes, Teeth Not Visible, Senior, Receding Hairline.

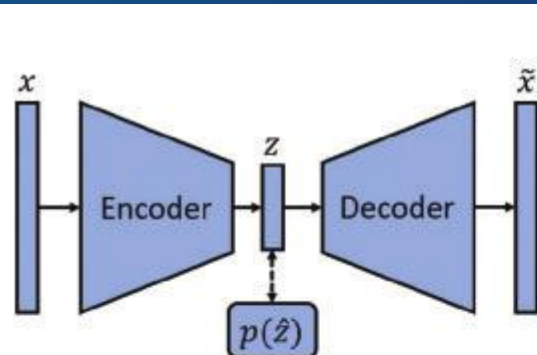
VAE



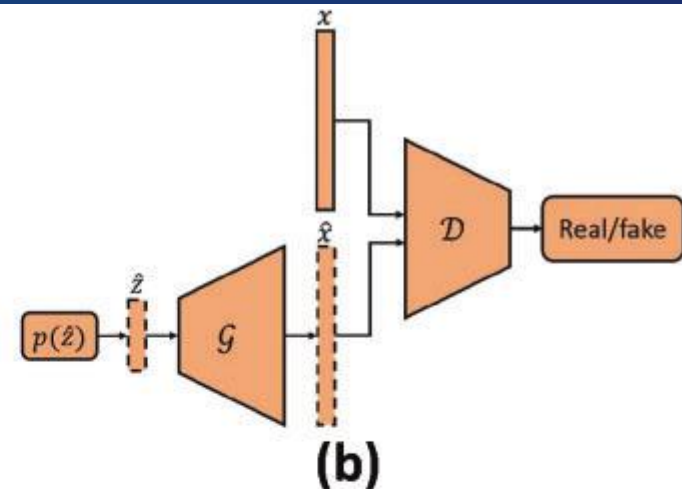
GAN



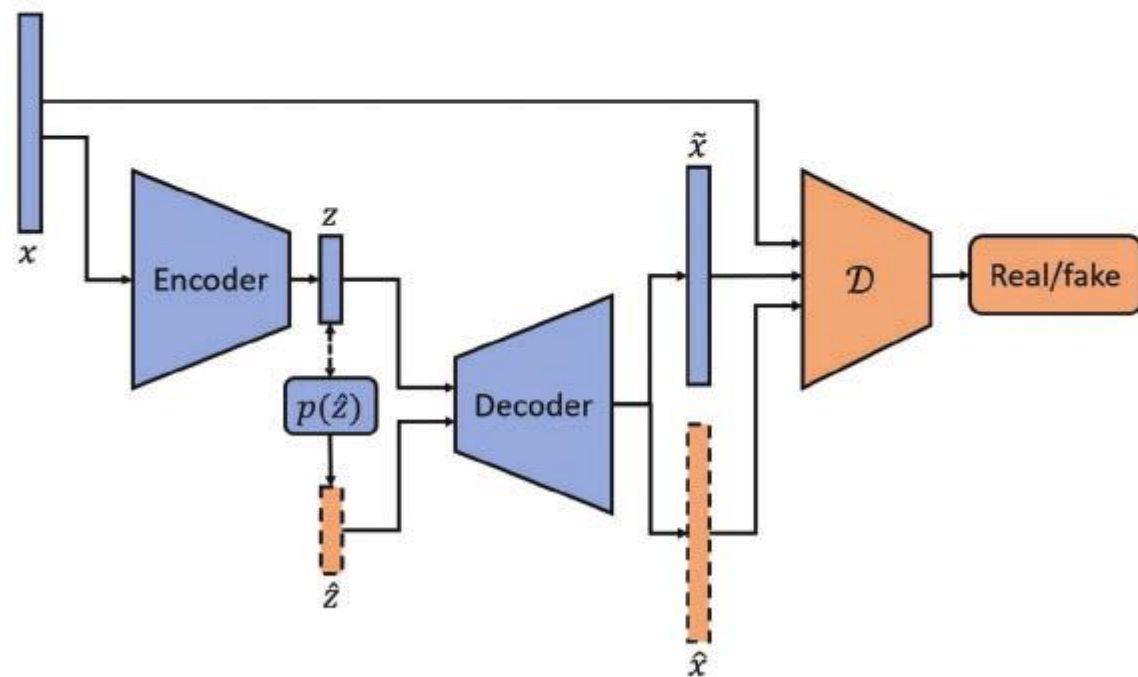
VAE/GAN



(a)



(b)



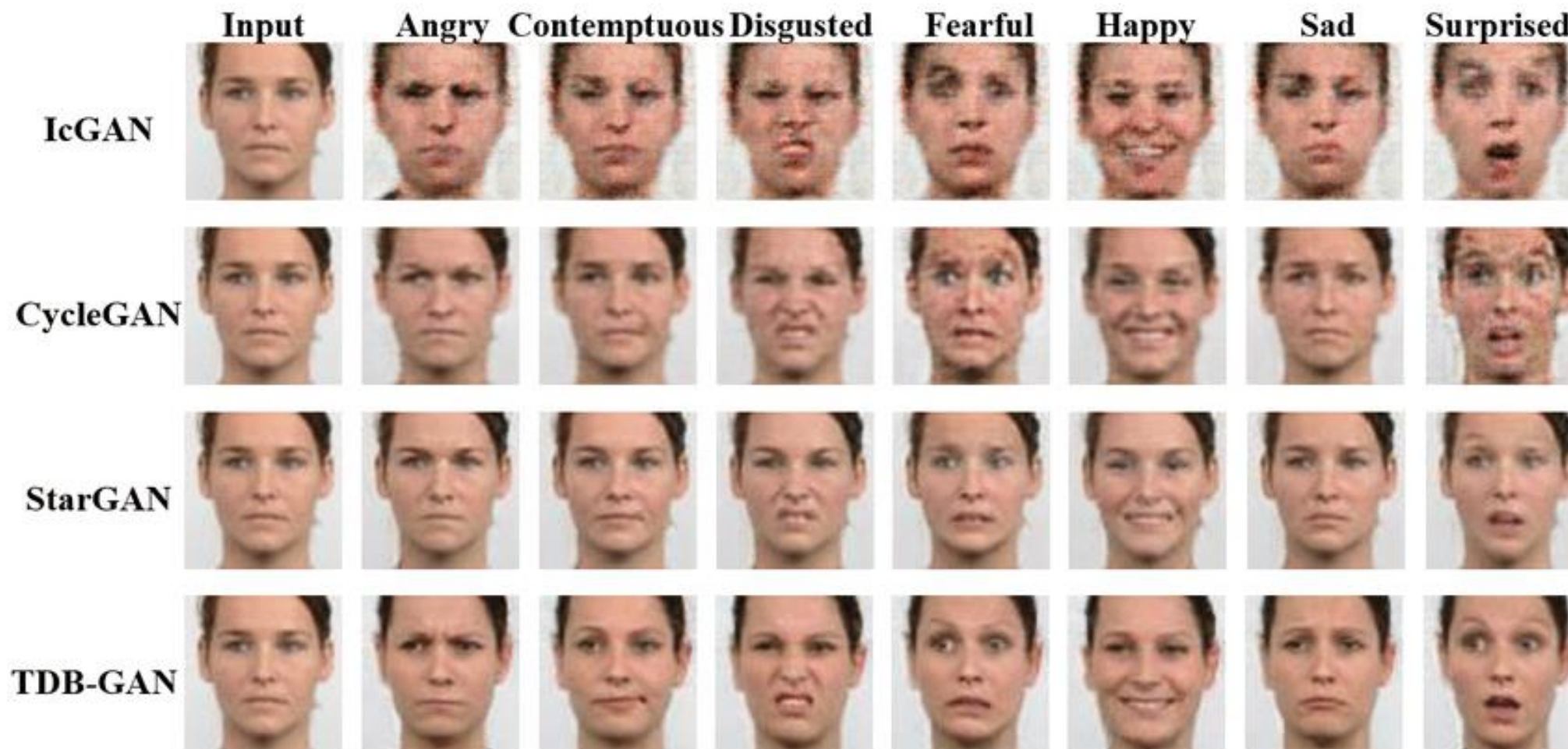
(c)



GAN Variants

- Conditional GAN: <https://arxiv.org/abs/1411.1784>
- Image to image translation: <https://arxiv.org/abs/1611.07004>
- Super-resolution: <https://arxiv.org/abs/1609.04802>
- Face inpainting: <https://arxiv.org/abs/1607.07539>
- Cycle GAN (style transfer): <https://arxiv.org/abs/1703.10593>
- Drag GAN (interactive editing): <https://github.com/XingangPan/DragGAN>

GAN applications – Image synthesis





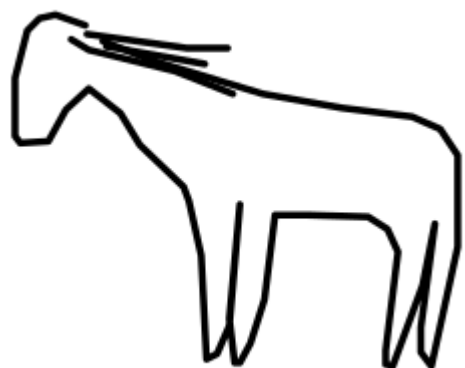
GAN applications – Sketch to Image



[Sketch Your Own GAN](#) - Paper

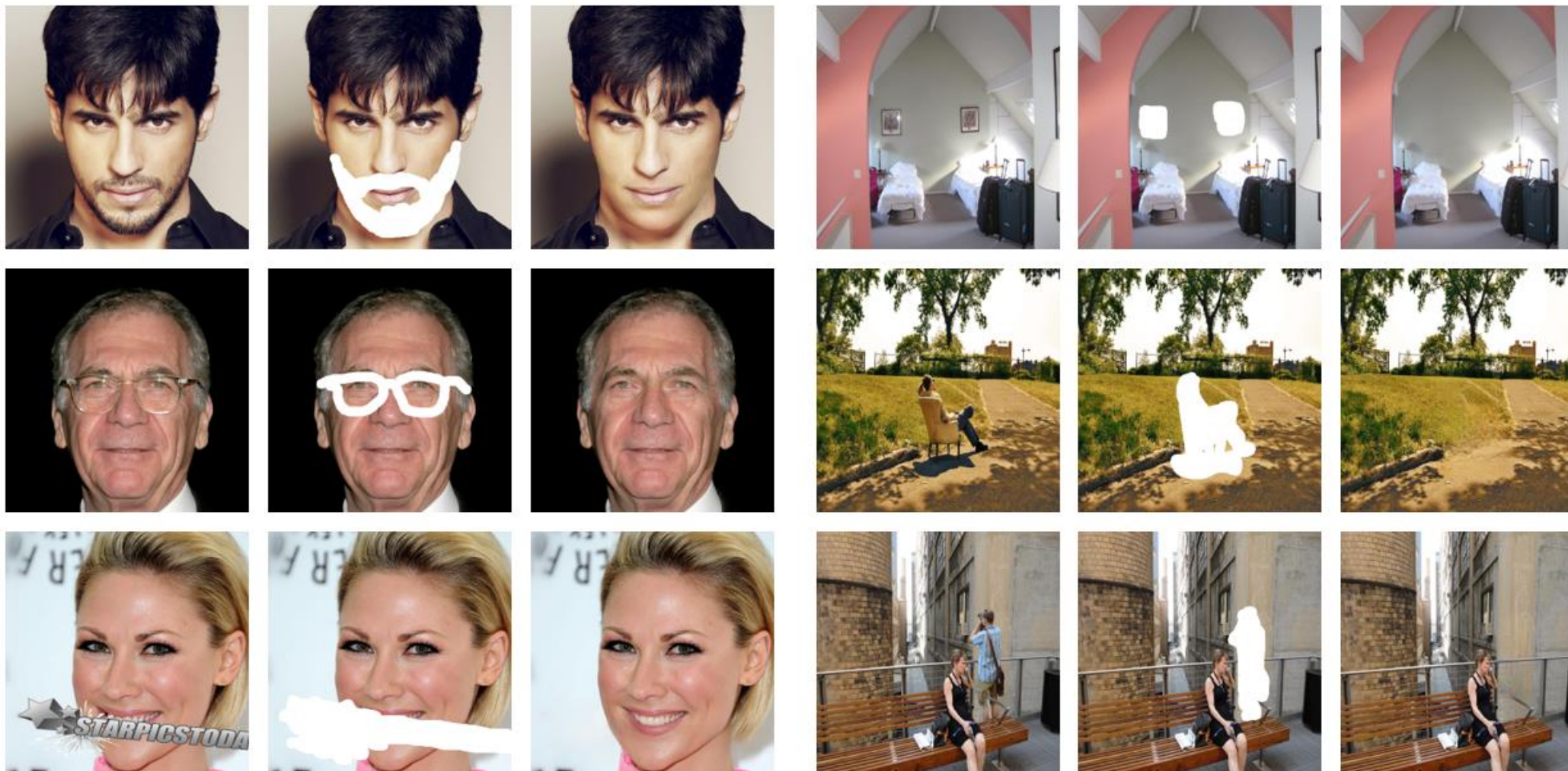


GAN applications – Sketch to Image





GAN applications – Image Inpainting



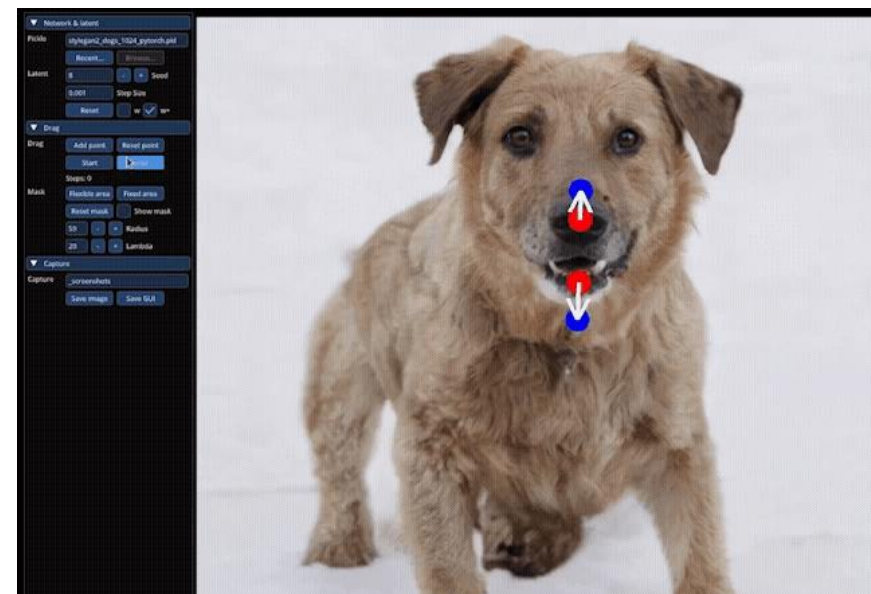
(a) Face Editing

(b) Object Removal

[Guide to AOT-GAN for High-Resolution Image Inpainting & Object Removal | Width.ai](#)



GAN applications – DragGAN



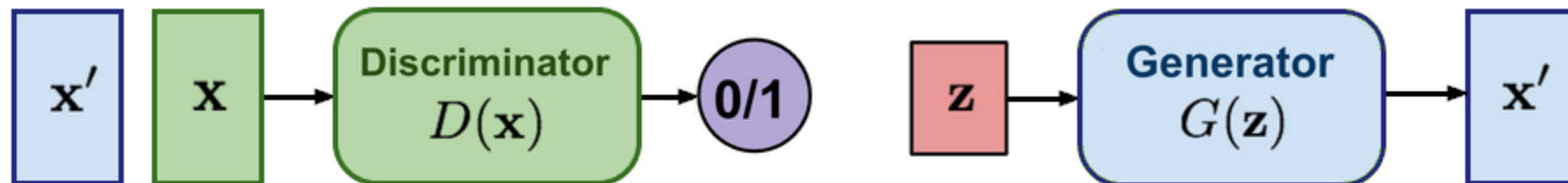


[GAN Lab: Play with Generative Adversarial Networks in Your Browser!](#)

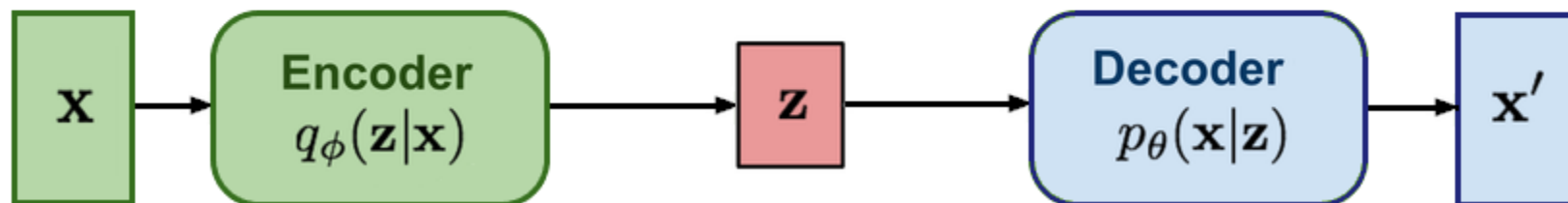


Diffusion Models

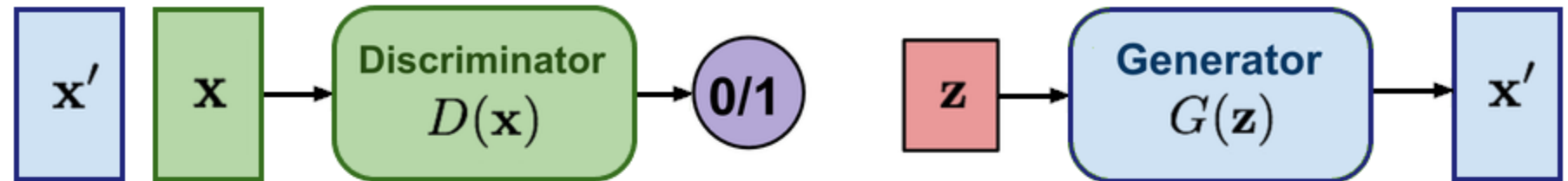
GAN: Adversarial training



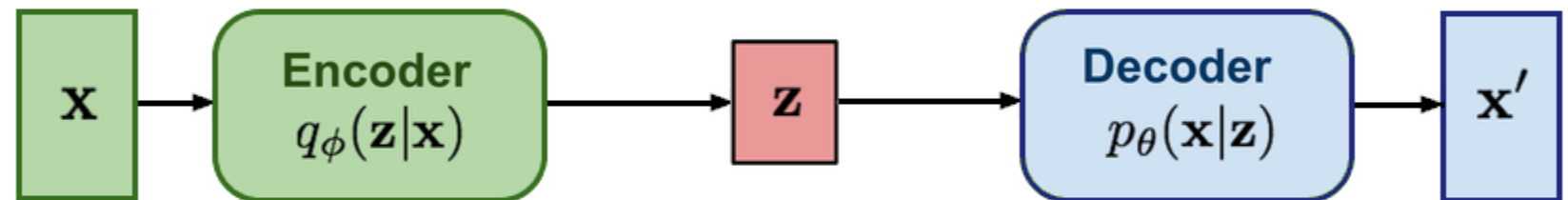
VAE: maximize variational lower bound



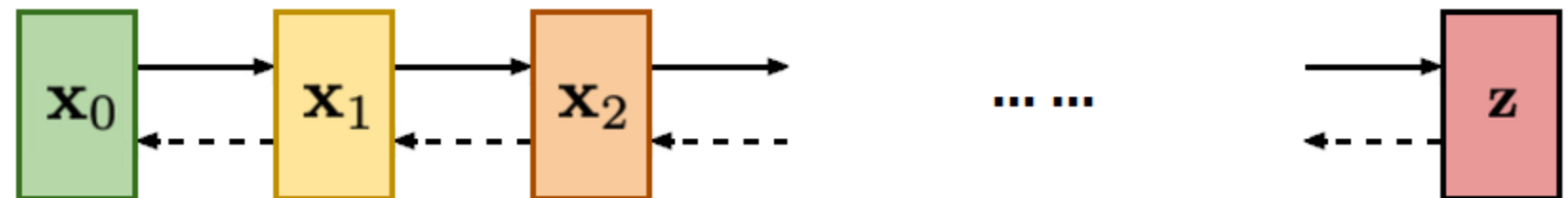
GAN: Adversarial training



VAE: maximize variational lower bound

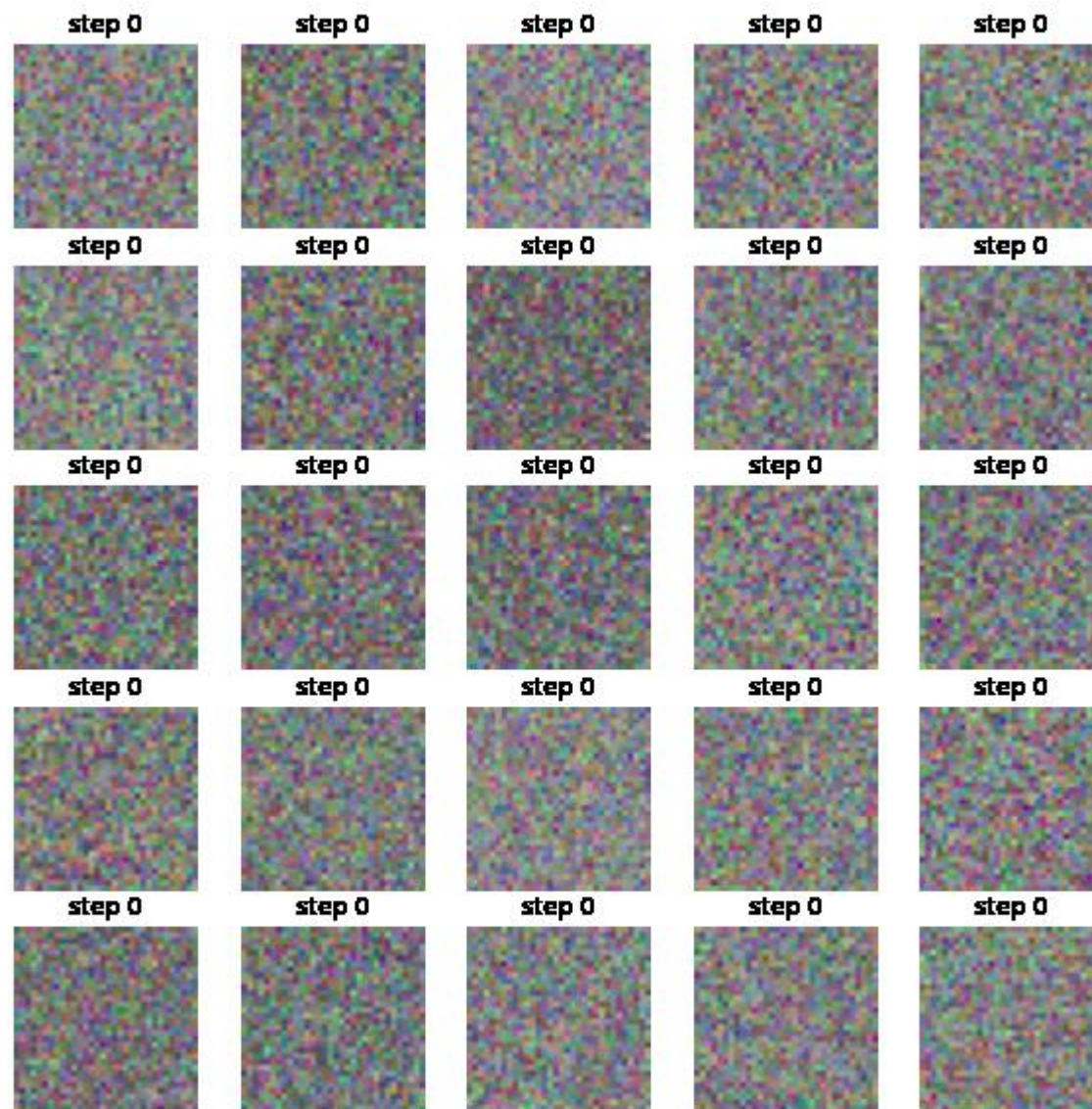


Diffusion models:
Gradually add Gaussian noise and then reverse





Diffusion Models



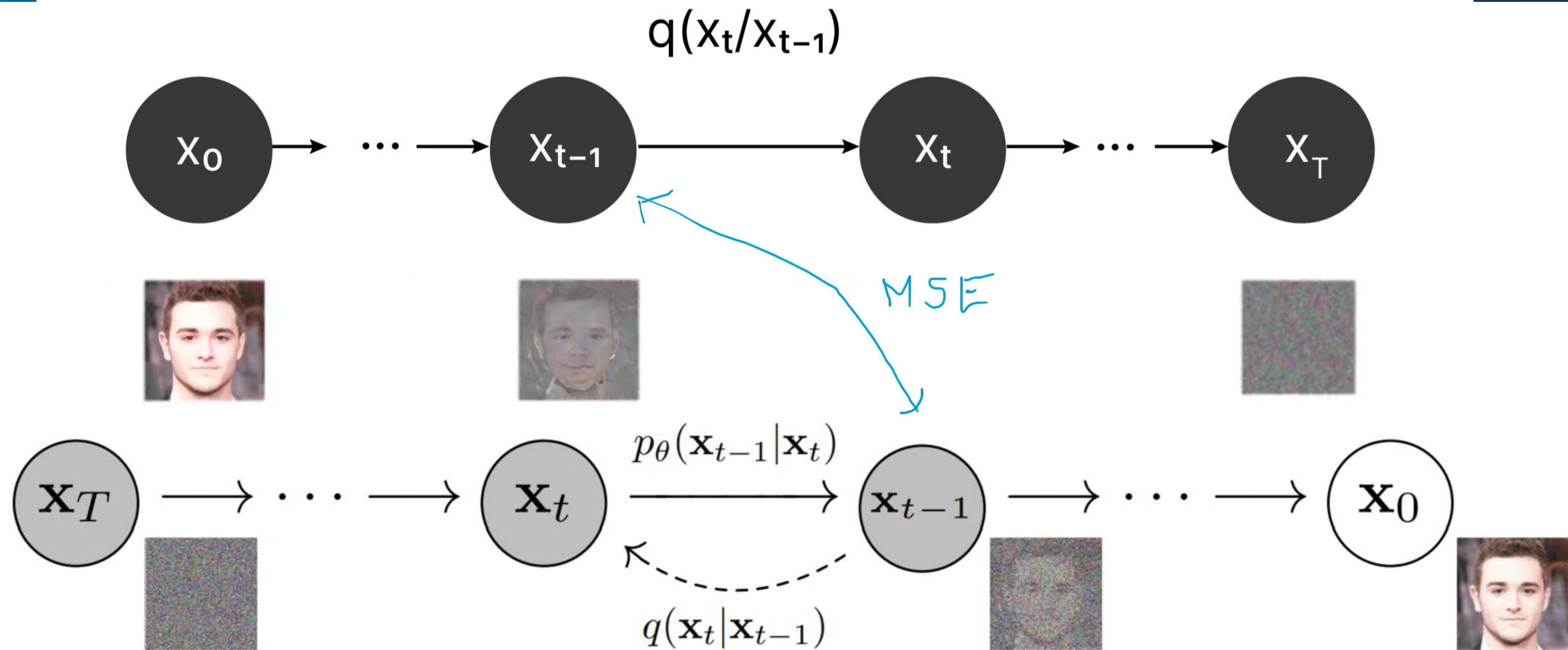


Diffusion Models





Diffusion Models





Advantages of Diffusion Models

- High-fidelity samples. It's due to the nature of gradually removing noise. Unlike VAEs and GANs, which generate samples at once, diffusion models create samples step by step. The model first creates a coarse image structure and then focuses on adding fine details on top.
- High diversity samples. Likelihood maximization covers all modes of the training dataset.
- The intermediate noisy images serve as latent codes and have the same size as the training images. This is one of the reasons why diffusion models can generate high-fidelity samples.
- Easy to train. It has one tractable likelihood loss.



Disadvantages of diffusion models

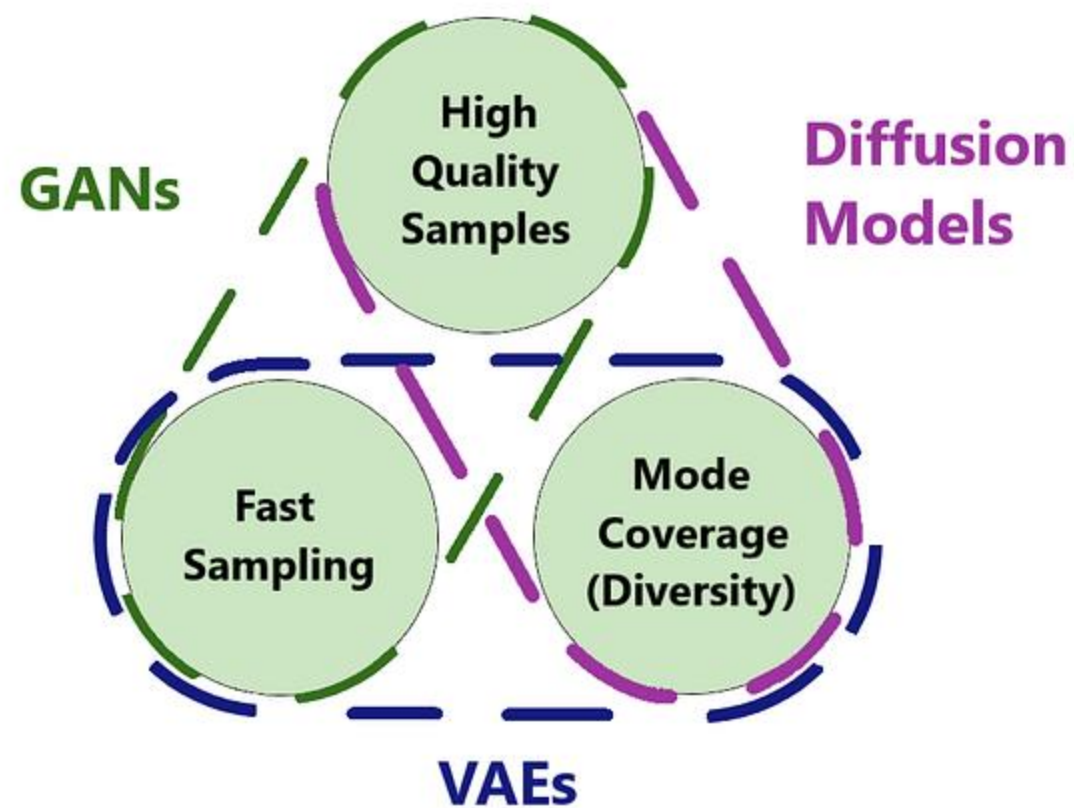
- **Slow generation speed:** The primary drawback is that generating a single sample is computationally expensive and slow because it requires hundreds or thousands of iterative denoising steps.
- **Higher computational resources:** The iterative process can require significant computational resources for both training and inference.



[Stable Diffusion AI - AI Image Generator \(Free, Unlimited\)](#)



Comparing all models





What will we learn in the next lecture?

- Reinforcement Learning
- Elements of RL
- Markov Decision Processes
- Q-Learning
- Deep Q-Learning
- Policy Gradients